Components

Components, interfaces, and integration

Components

A component is a distinct part of a system that performs a *specific responsibility* and interacts with other parts through defined interfaces.

Rather than building a system as *one large block of code*, developers divide functionality into smaller, manageable units called components.

A component may be:

- A software module
- A database
- A hardware device
- A cloud service
- A third-party application

Examples

Each component focuses on doing *one job well* while collaborating with other components to achieve overall system goals.

Online Store:

- Product Catalog
- Shopping Cart
- Payment Processing
- User Authentication
- Order Management

University System:

- Student Records
- Enrollment System
- Grading System
- Payment System
- Learning Management System

Why Systems Use Components

Large systems quickly become difficult to manage when everything is built as a *single unit*.

Breaking systems into components provides several advantages.

Separation of Responsibilities

Each component focuses *on a specific task*.

Example:

- Authentication handles login
- Inventory manages products
- Payment processes transactions

This makes systems easier to *understand and maintain*.

Easier Development

Multiple teams can develop *different components* simultaneously.

Example:

- Frontend team develops the website
- Backend team develops APIs
- Database team manages data storage

Reusability

Components can be reused *across projects*.

Example: A login component may be used by:

- Web applications
- Mobile applications
- Internal company systems

Maintainability

Issues can often be fixed in one component *without affecting* others.

Example: A payment component can be updated without redesigning the entire application.

Reliability

Failures can often be *isolated*.

Example: If the recommendation system fails, customers may still be able to browse and purchase products.

Categories of Components

Most information systems contain *multiple categories* of components.

- Software Components - Programs and services that provide system functionality.
- Hardware Components - Physical devices that host or support system operations.
- Third-Party Components - External services integrated into the system.

Software Components

Software components contain executable logic that performs business functions.

They are the most visible part of most information systems.

Examples

Frontend Components

Responsible for user interaction.

Examples:

- Web pages
- Mobile app screens
- Dashboards

Backend Components

Responsible for business logic and processing.

Examples:

- Authentication service
- Inventory service
- Reporting service

Data Components

Responsible for storing information.

Examples:

- Databases
- Data warehouses
- File storage systems

Hardware Components

Hardware components provide the physical infrastructure required by software.

Without hardware, software cannot execute.

Examples

Computing Devices

- Servers
- Desktop Computers
- Laptops
- Smartphones

Networking Equipment

- Routers
- Switches
- Firewalls

Peripheral Devices

- Printers
- Scanners
- Card Readers

Specialized Devices

- Sensors
- Cameras
- IoT Devices

Third-Party Components

Organizations often use components developed by external vendors instead of building everything themselves.

Why Use Third-Party Components?

Building every feature internally can be expensive and time-consuming.

Third-party services allow organizations to:

- Reduce development time
- Lower costs
- Access specialized functionality
- Improve reliability

However, these also carry certiain risks like:

- Vendor dependency
- Service outages
- API changes
- Subscription costs
- Security concerns

Examples

Payment Services

- Stripe
- PayPal
- GCash APIs

Communication Services

- Twilio
- SendGrid

Cloud Services

- AWS
- Microsoft Azure
- Google Cloud

Authentication Services

- Google Login
- Microsoft Login
- Auth0

Interfaces

Components rarely operate alone.

They *must communicate* with each other.

An interface defines the rules governing this communication.

Think of an interface as a *contract* between components.

The contract specifies:

- What requests can be made
- What information must be provided
- What results will be returned
- How errors are handled

This allows for modularity

Interfaces

A restaurant menu is an interface.

Customers:

- See available options
- Place orders using defined choices

Customers do not need to *know how food* is prepared.

Similarly, components use interfaces *without knowing internal implementation details*.

APIs

Application Programming Interface

An API is the most common interface used in software systems.

APIs allow applications and services to communicate programmatically.

API Communication Process

1. Client sends request
2. Server receives request
3. Processing occurs
4. Response is returned

Example

Mobile Banking App

Request: "Check account balance"

API Processes:

- Validate user
- Retrieve account data

Response: Current balance information

Common API Types

REST APIs

Most common web API style.

Uses:

- HTTP
- JSON

GraphQL APIs

Allows clients to request only needed data.

SOAP APIs

Older enterprise integration technology.

Often used in legacy systems.

API Example

Food Delivery Application

Components:

- Mobile App
- Restaurant System
- Payment Gateway
- Delivery Tracking System

Communication:

Mobile App → Restaurant API

Restaurant API → Order Database

Mobile App → Payment API

Tracking System → Location API

Each interaction occurs through APIs.

Other Interfaces

Not all interfaces are APIs.

Several technologies help systems communicate.

SDK (Software Development Kit)

A collection of tools, libraries, documentation, and sample code used to build applications.

Purpose: Makes integration easier.

Examples:

- Android SDK
- Firebase SDK
- Facebook SDK

Benefits:

- Faster development
- Less coding
- Standardized integration

Drivers

Drivers are specialized software that allow operating systems to communicate with hardware.

Without drivers:

- Hardware may not function properly
- Operating systems cannot understand device instructions

Examples:

- Printer Drivers
- Graphics Drivers
- Audio Drivers
- Network Drivers

Drivers act as translators between hardware and software.

Integration

Integration

Modern systems consist of many components.

These components must work together to provide complete functionality.

Integration is the process of connecting components, systems, or services so they can exchange data and coordinate operations.

Goals of Integration

- Share information
- Automate processes
- Reduce duplicate work
- Improve efficiency
- Create seamless user experiences

Types of Integration

Integration is generally classified into two categories:

- Internal Integration

Connecting components within the same organization.

- External Integration

Connecting systems with outside organizations or services.

Internal Integration

Internal integration connects systems that belong to the same organization.

Examples

University

- Enrollment System ↔ Student Records
- Student Records ↔ Billing System
- Billing System ↔ Accounting System

Hospital

- Patient Management System
- Laboratory System
- Pharmacy System
- Billing System

Benefits

- Centralized information
- Consistent data
- Faster workflows
- Better reporting

External Integration

External integration connects systems belonging to different organizations.

Examples

E-Commerce

- Payment Gateway
- Shipping Provider
- SMS Provider
- Email Provider

Banking

- ATM Networks
- Payment Networks
- Government Verification Systems

Benefits

- Expanded capabilities
- Access to external services
- Improved customer experience

Challenges

- Security risks
- Compatibility issues
- Network failures
- Third-party dependencies

Real-World Example: Online Shopping System

Components

Frontend:

- Website
- Mobile Application

Backend:

- Product Service
- Inventory Service
- Order Service

Data:

- Product Database
- Customer Database

Third Party:

- Payment Gateway
- SMS Service
- Email Service

How Components Communicate

Customer Places Order

1. Website sends request to Order API
2. Order Service checks Inventory Service
3. Inventory Service checks Database
4. Order Service requests Payment Gateway
5. Payment Gateway confirms payment
6. Email Service sends confirmation
7. SMS Service sends notification

Multiple independent components cooperate to complete one business process.